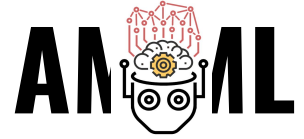




**Politecnico
di Torino**



Advanced Machine Learning

A.A. 2025/2026

Tatiana Tommasi

Some other (CV) Tasks

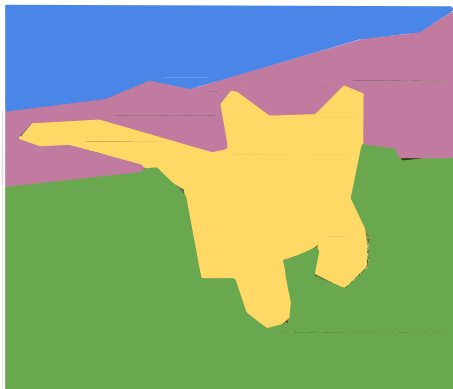
Classification



CAT

Single Object

Semantic Segmentation



GRASS, CAT,
TREE, SKY

No objects, just pixels

Object Detection



DOG, DOG, CAT

Multiple Object

Instance Segmentation



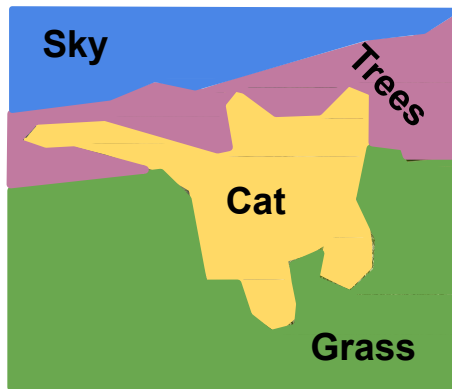
DOG, DOG, CAT

[This image is CC0 public domain](#)

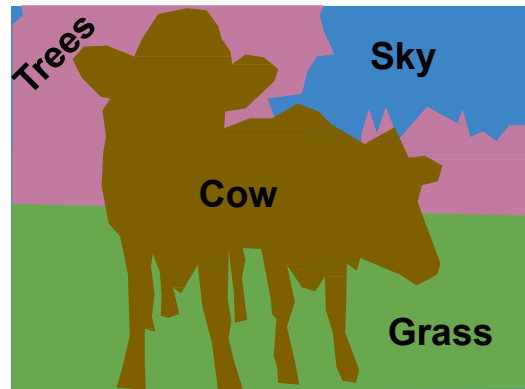
Semantic Segmentation

Label each pixel in the image with a category label

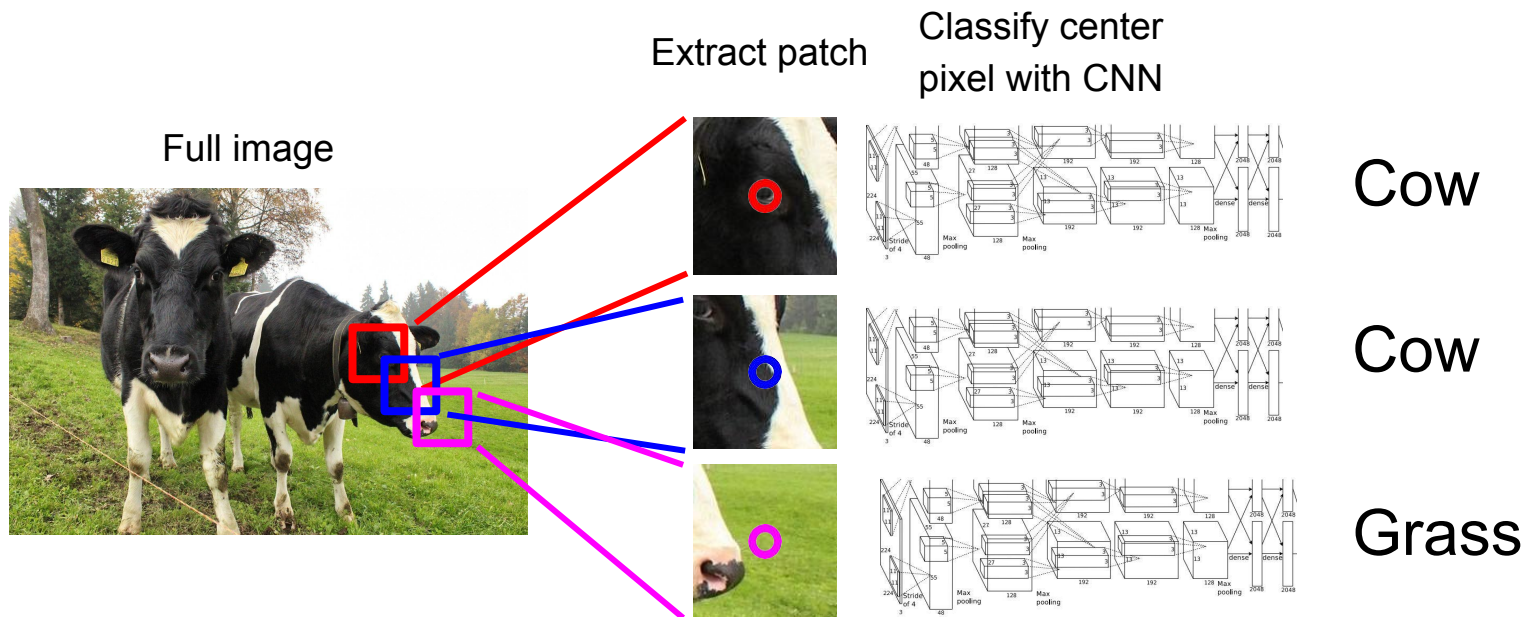
Don't differentiate instances, only care about pixels



This image is CC0 public domain

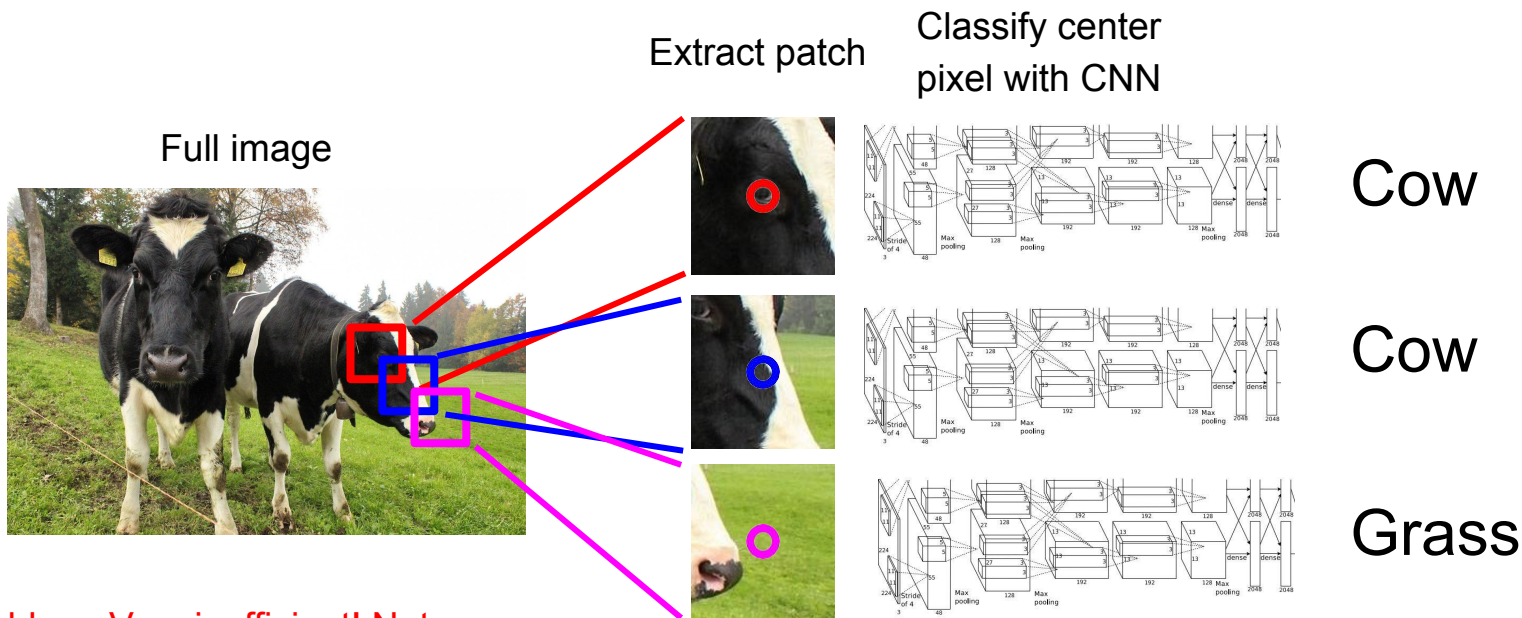


Semantic Segmentation Idea: Sliding Window



Farabet et al, "Learning Hierarchical Features for Scene Labeling," TPAMI 2013
Pinheiro and Collobert, "Recurrent Convolutional Neural Networks for Scene Labeling", ICML 2014

Semantic Segmentation Idea: Sliding Window

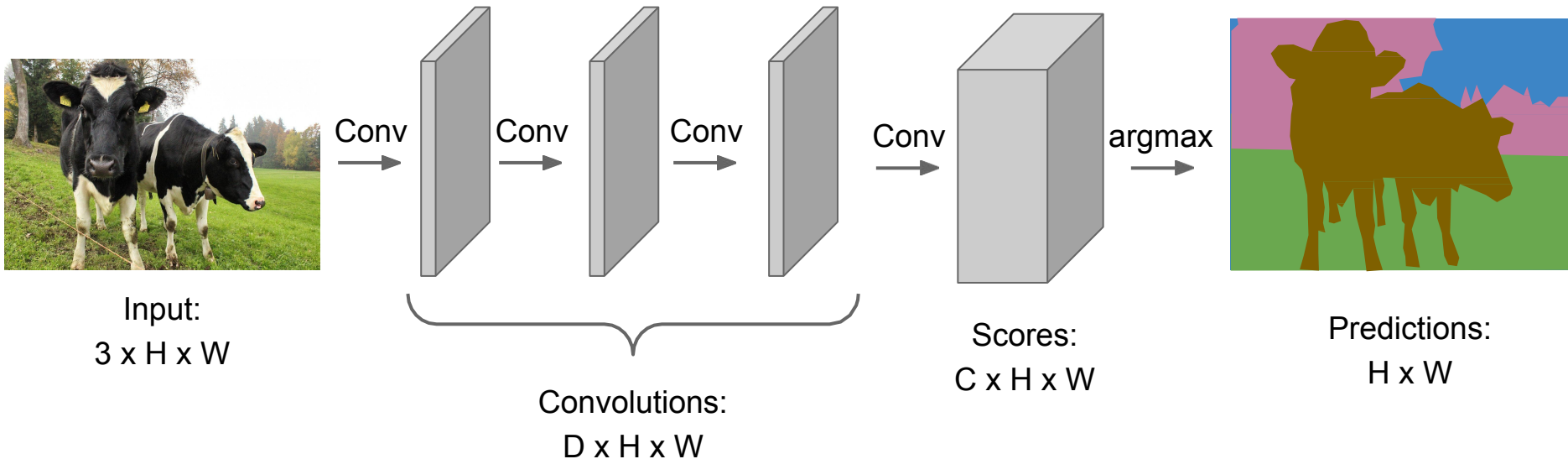


Problem: Very inefficient! Not reusing shared features between overlapping patches

Farabet et al, "Learning Hierarchical Features for Scene Labeling," TPAMI 2013
Pinheiro and Collobert, "Recurrent Convolutional Neural Networks for Scene Labeling", ICML 2014

Semantic Segmentation Idea: Fully Convolutional

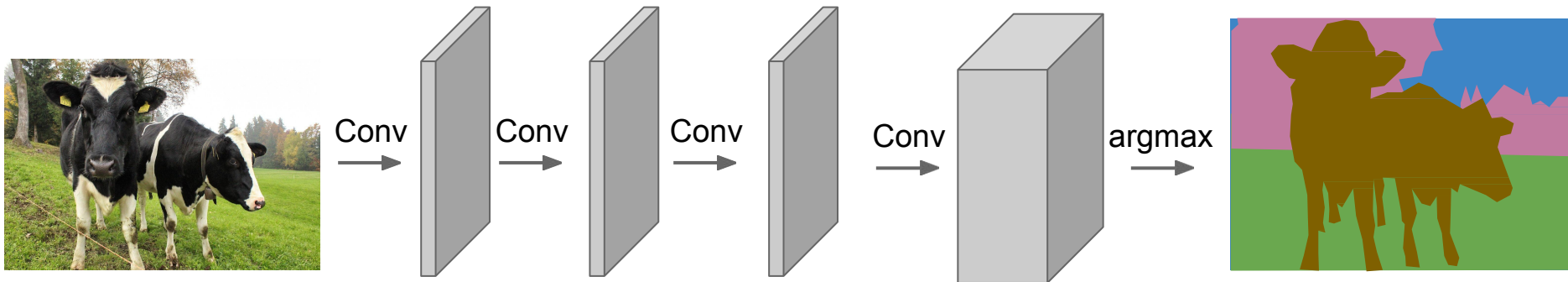
Design a network as a bunch of convolutional layers to make predictions for pixels all at once!



Loss function: Per-Pixel cross-entropy

Semantic Segmentation Idea: Fully Convolutional

Design a network as a bunch of convolutional layers to make predictions for pixels all at once!



Input:
 $3 \times H \times W$

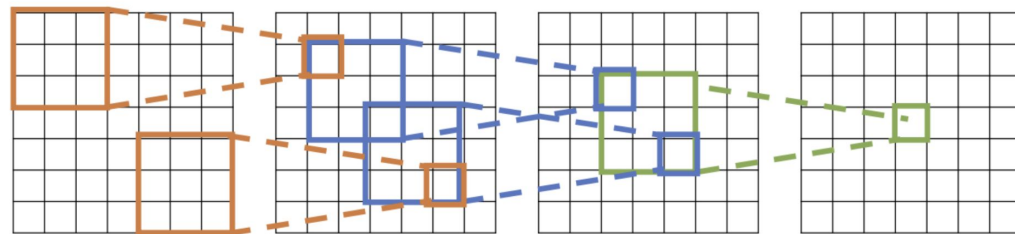
Problem #1: Effective receptive field size is linear in number of conv layers: With L 3×3 conv layers, receptive field is $1+2L$

Problem #2: Convolution on high res images is expensive! Recall ResNet stem aggressively downsamples

Receptive Fields

Each successive convolution adds $K - 1$ to the receptive field size

With L layers the receptive field size is $1 + L * (K - 1)$



Input

Output

Be careful – “receptive field in the input” vs “receptive field in the previous layer”
Hopefully clear from context!



Slide credit: Justin Johnson

75

two 3x3 conv -- look at the content of a **5x5** region of the input

three 3x3 conv -- look at the content of a **7x7** region of the input

We need a lot of layers to have a very big receptive field size and be able to assign a label to a pixel while taking into consideration the shape of the whole object...

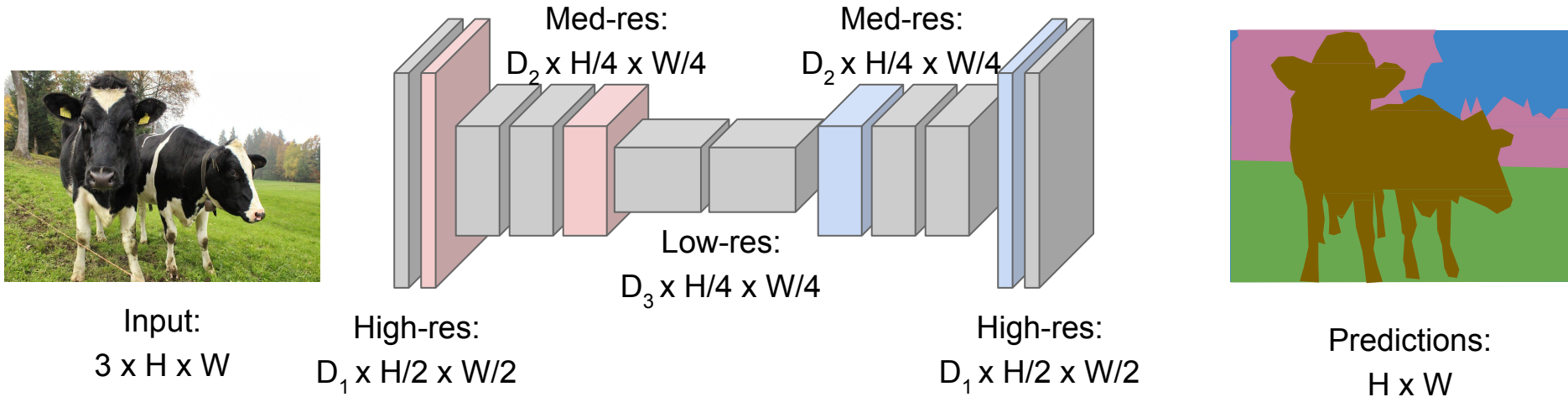
Problem #1: Effective receptive field size is linear in number of conv layers: With L 3x3 conv layers, receptive field is $1+2L$



Slide credit: Fei Fei Li

Semantic Segmentation Idea: Fully Convolutional

Design network as a bunch of convolutional layers, with **downsampling** and **upsampling** inside the network!



Long, Shelhamer, and Darrell, "Fully Convolutional Networks for Semantic Segmentation", CVPR 2015
Noh et al, "Learning Deconvolution Network for Semantic Segmentation", ICCV 2015

Semantic Segmentation Idea: Fully Convolutional

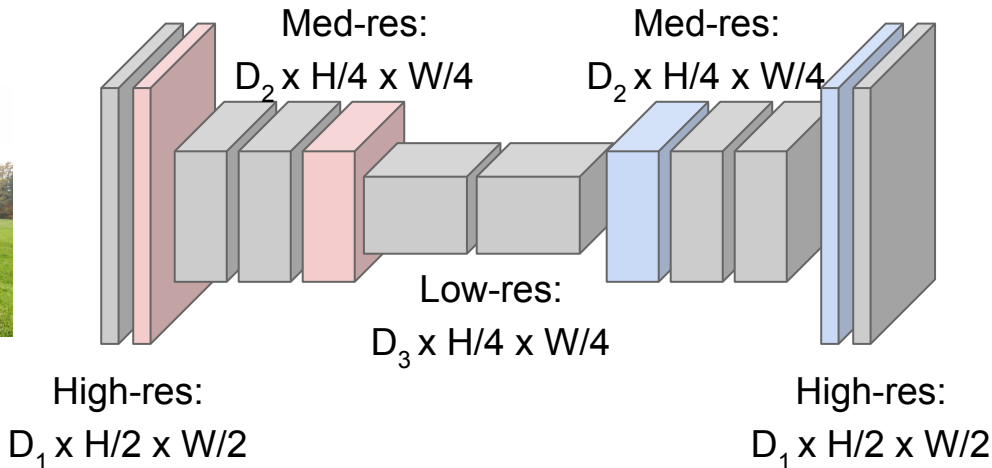
Downsampling:
Pooling, strided
convolution

Design network as a bunch of convolutional layers, with
downsampling and **upsampling** inside the network!

Upsampling:
???



Input:
 $3 \times H \times W$



Predictions:
 $H \times W$

Long, Shelhamer, and Darrell, "Fully Convolutional Networks for Semantic Segmentation", CVPR 2015
Noh et al, "Learning Deconvolution Network for Semantic Segmentation", ICCV 2015

In-Network Upsampling: “Unpooling”

Bed of Nails

1	2
3	4



1	0	2	0
0	0	0	0
3	0	4	0
0	0	0	0

Input
 $C \times 2 \times 2$

Output
 $C \times 4 \times 4$

Nearest Neighbor

1	2
3	4



1	1	2	2
1	1	2	2
3	3	4	4
3	3	4	4

Input
 $C \times 2 \times 2$

Output
 $C \times 4 \times 4$

In-Network Upsampling: Bilinear Interpolation

1	•	•	2
•	•	•	•
•	•	•	•
3	•	•	4



1.00	1.25	1.75	2.00
1.50	1.75	2.25	2.50
2.50	2.75	3.25	3.50
3.00	3.25	3.75	4.00

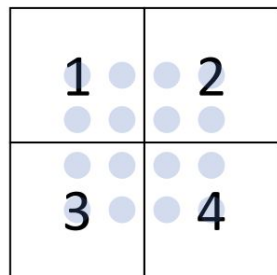
Input: C x 2 x 2

Output: C x 4 x 4

$$f_{x,y} = \sum_{i,j} f_{i,j} \max(0, 1 - |x - i|) \max(0, 1 - |y - j|) \quad \begin{array}{l} i \in \{\lfloor x \rfloor - 1, \dots, \lceil x \rceil + 1\} \\ j \in \{\lfloor y \rfloor - 1, \dots, \lceil y \rceil + 1\} \end{array}$$

Use two closest neighbors in x and y
to construct linear approximations

In-Network Upsampling: Bicubic Interpolation



0.68	1.02	1.56	1.89
1.35	1.68	2.23	2.56
2.44	2.77	3.32	3.65
3.11	3.44	3.98	4.32

Input: $C \times 2 \times 2$

Output: $C \times 4 \times 4$

Use **three** closest neighbors in x and y to construct **cubic** approximations
(This is how we normally resize images!)

In-Network Upsampling: “Max Unpooling”

Max Pooling: Remember which position had the max

1	2	6	3
3	5	2	1
1	2	2	1
7	3	4	8



5	6
7	8



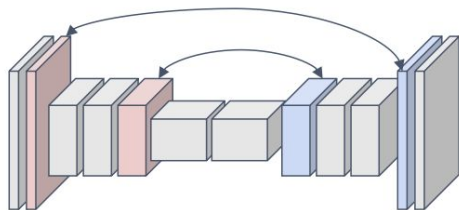
Rest
of
net



1	2
3	4



0	0	2	0
0	1	0	0
0	0	0	0
3	0	0	4

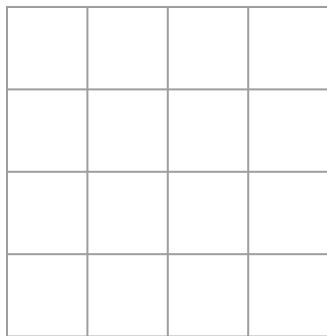


Pair each downsampling layer
with an upsampling layer

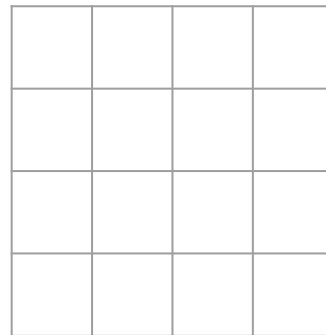
Noh et al, “Learning Deconvolution Network for Semantic Segmentation”, ICCV 2015

Learnable upsampling: Transpose Convolution

Recall: Typical 3 x 3 convolution, stride 1 pad 1



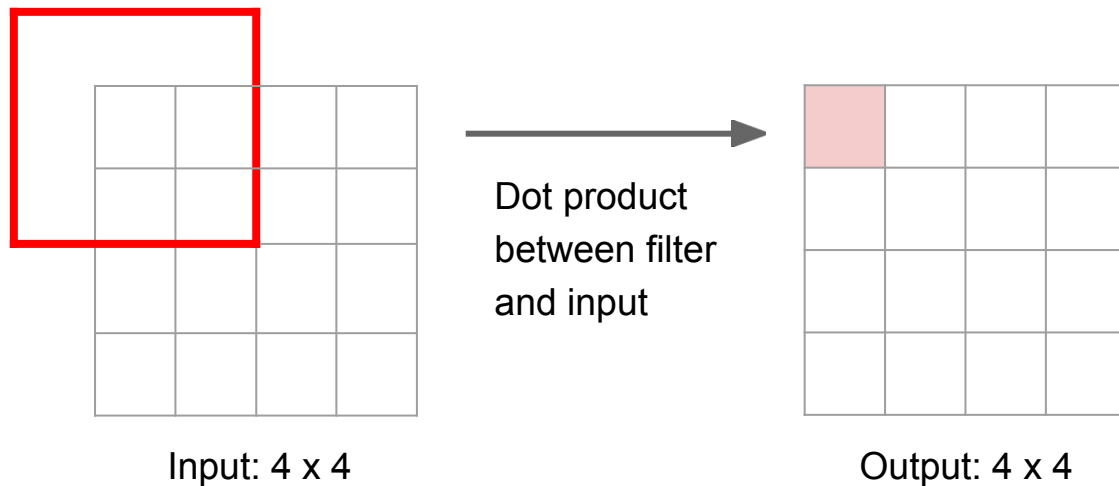
Input: 4 x 4



Output: 4 x 4

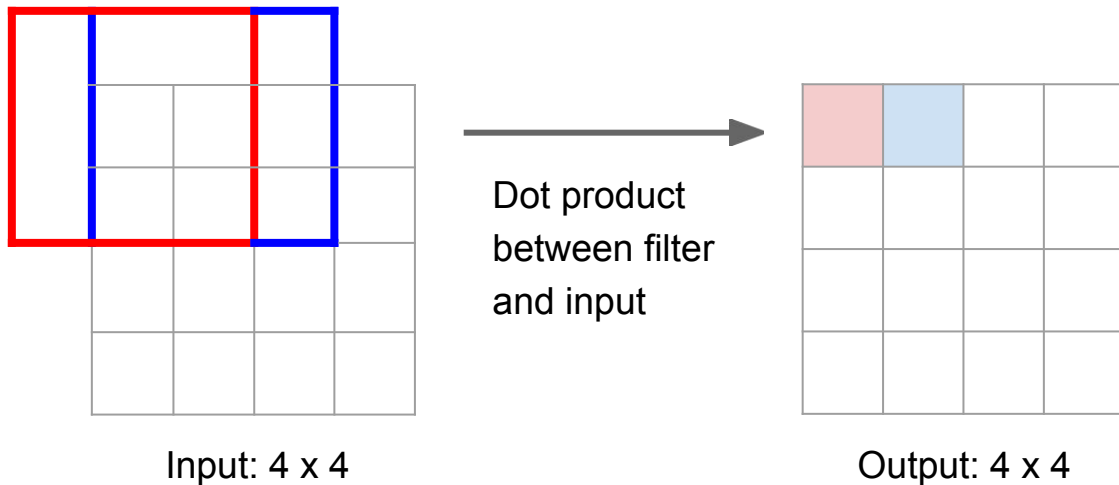
Learnable upsampling: Transpose Convolution

Recall: Normal 3 x 3 convolution, stride 1 pad 1



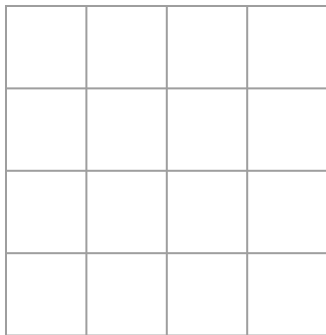
Learnable upsampling: Transpose Convolution

Recall: Normal 3 x 3 convolution, stride 1 pad 1

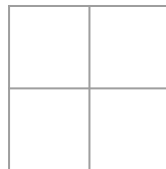


Learnable upsampling: Transpose Convolution

Recall: Normal 3 x 3 convolution, stride 2 pad 1



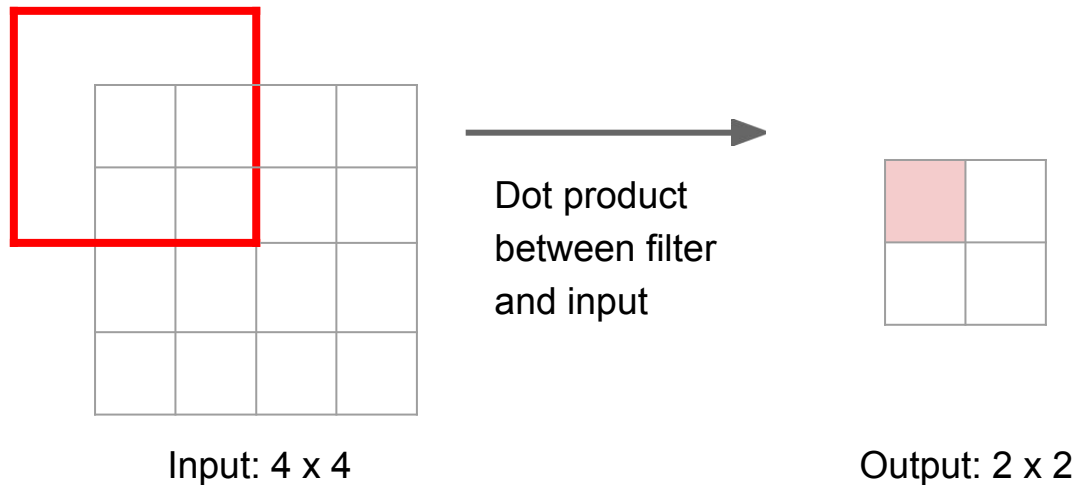
Input: 4 x 4



Output: 2 x 2

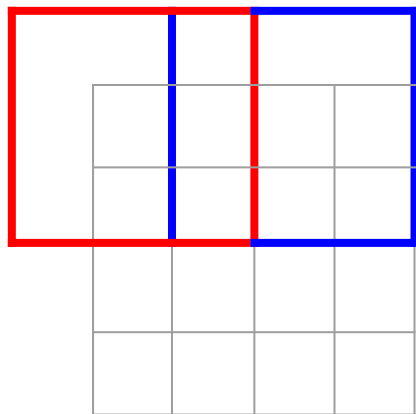
Learnable upsampling: Transpose Convolution

Recall: Normal 3 x 3 convolution, stride 2 pad 1



Learnable upsampling: Transpose Convolution

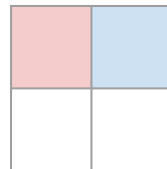
Recall: Normal 3 x 3 convolution, stride 2 pad 1



Input: 4 x 4



Dot product
between filter
and input



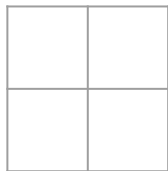
Output: 2 x 2

Filter moves 2 pixels in
the input for every one
pixel in the output

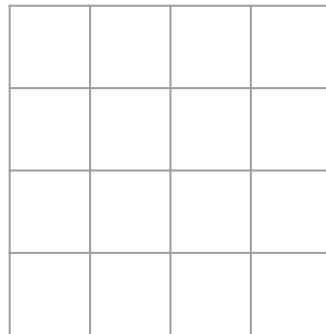
Stride gives ratio between
movement in input and
output

Learnable upsampling: Transpose Convolution

3 x 3 **transpose** convolution, stride 2 pad 1



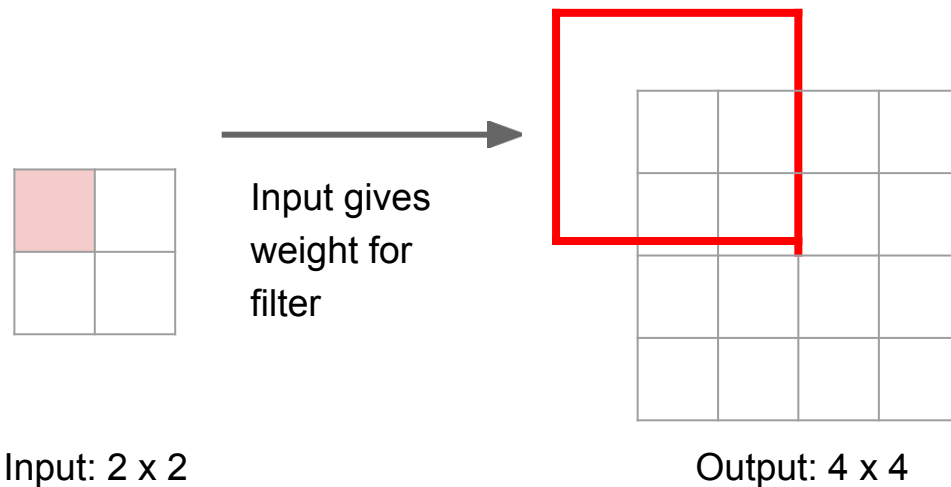
Input: 2 x 2



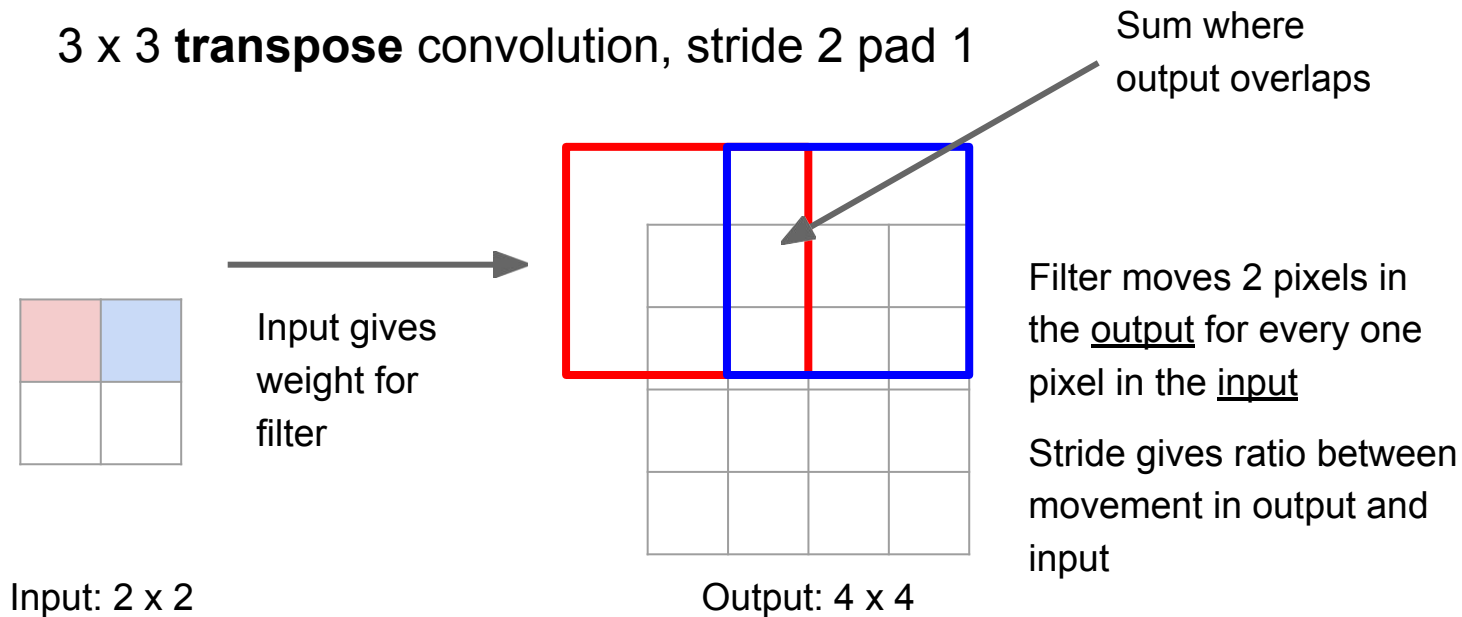
Output: 4 x 4

Learnable upsampling: Transpose Convolution

3 x 3 **transpose** convolution, stride 2 pad 1



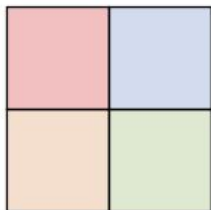
Learnable upsampling: Transpose Convolution



Learnable upsampling: Transpose Convolution

3 x 3 **convolution transpose**, stride 2

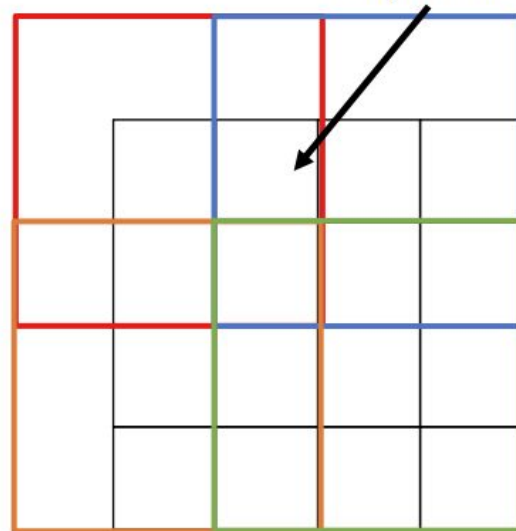
This gives 5x5 output – need to trim one pixel from top and left to give 4x4 output



Input: 2 x 2

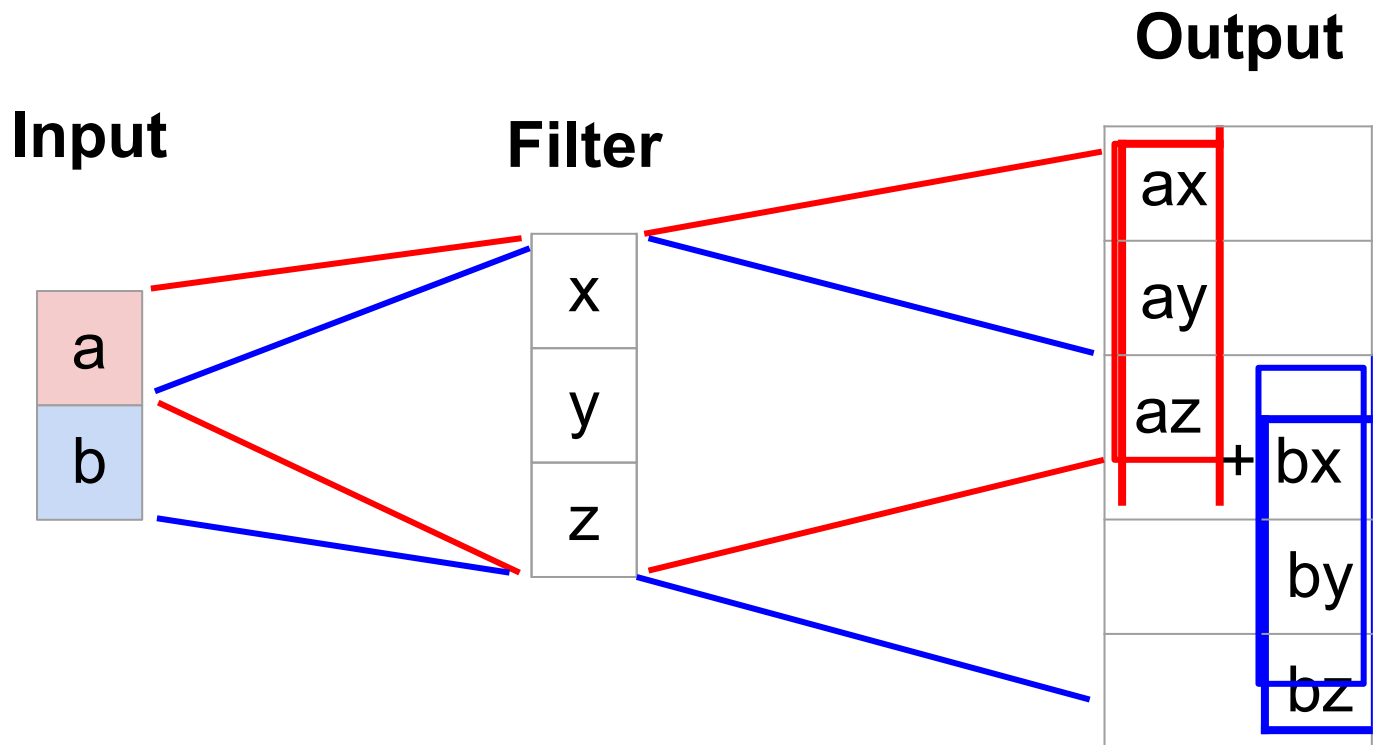
→
Weight filter by
input value and
copy to output

Sum where
output overlaps



Output: 4 x 4

Transpose Convolution: 1D Example

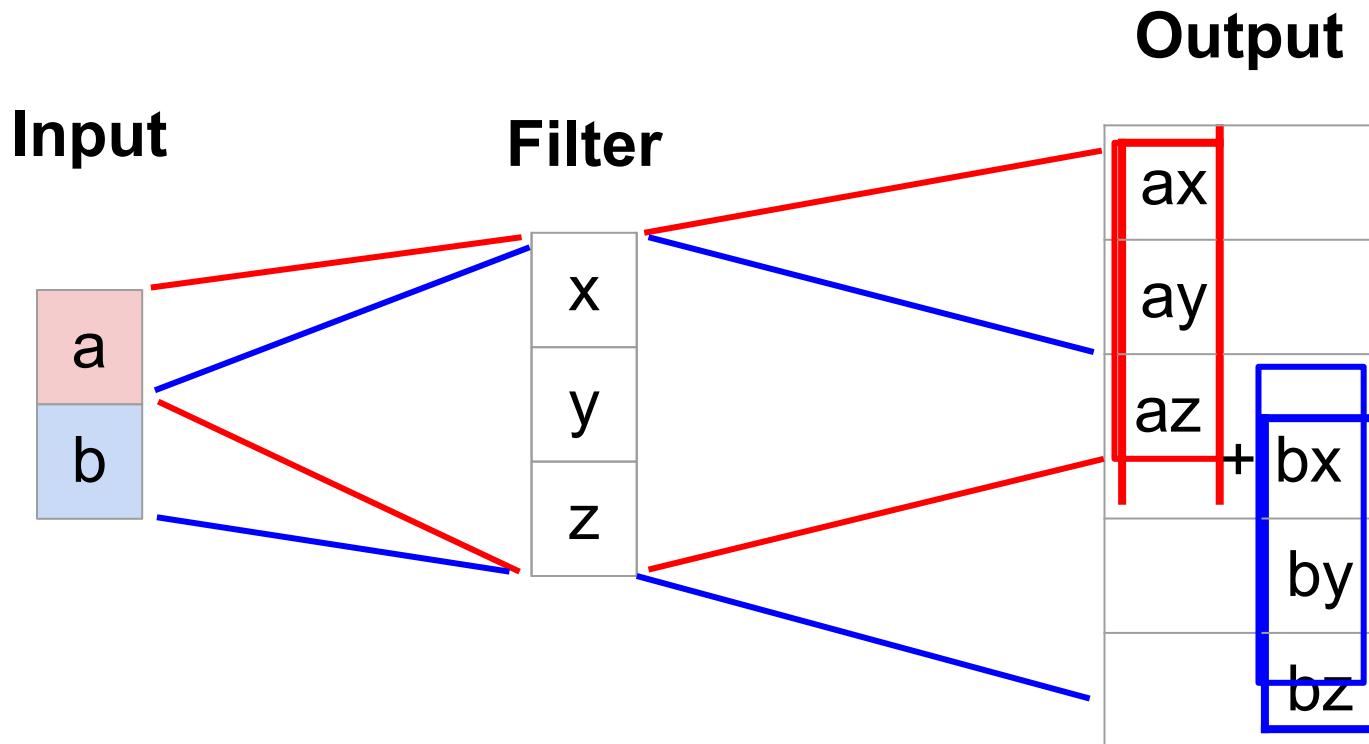


Output contains copies of the filter weighted by the input, summing at where it overlaps in the output

Need to crop one pixel from output to make output exactly 2x input

also check: http://d2l.ai/chapter_computer-vision/transposed-conv.html

Transpose Convolution: 1D Example



Other names:

- Deconvolution (bad)
- Upconvolution
- Fractionally strided convolution
- Backward strided convolution

also check: http://d2l.ai/chapter_computer-vision/transposed-conv.html

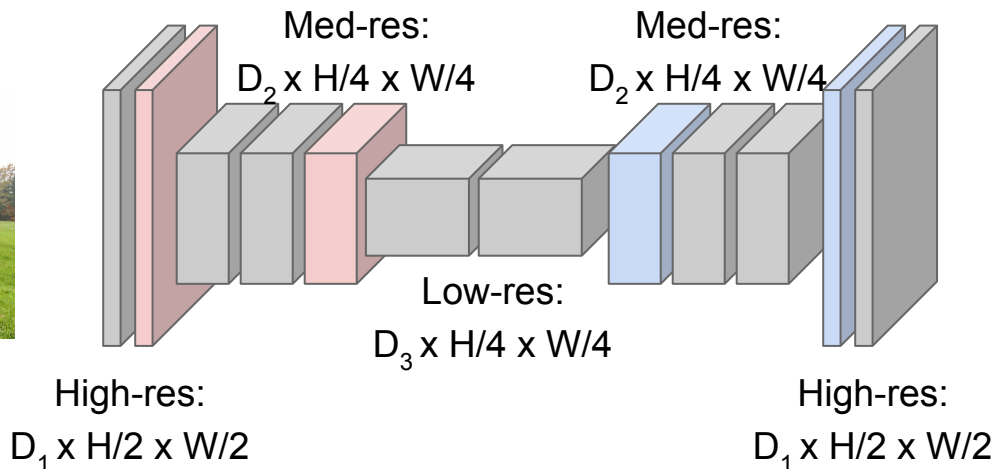
Semantic Segmentation Idea: Fully Convolutional

Downsampling:
Pooling, strided
convolution



Input:
 $3 \times H \times W$

Design network as a bunch of convolutional layers, with **downsampling** and **upsampling** inside the network!



Upsampling:
Unpooling or strided
transpose convolution



Predictions:
 $H \times W$

Long, Shelhamer, and Darrell, "Fully Convolutional Networks for Semantic Segmentation", CVPR 2015
Noh et al, "Learning Deconvolution Network for Semantic Segmentation", ICCV 2015

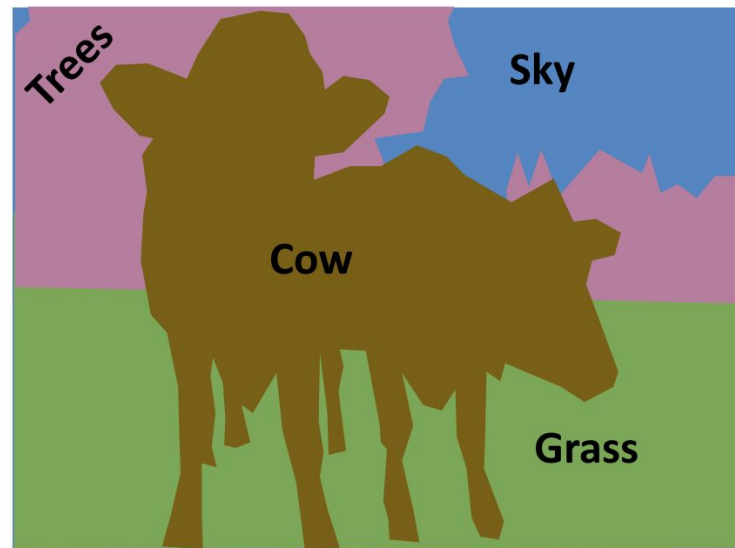
Loss function: Per-Pixel cross-entropy

Computer Vision Tasks

Object Detection: Detects individual object instances, but only gives box

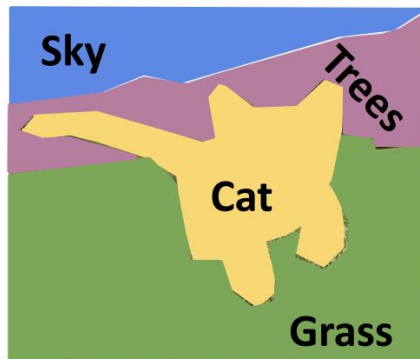


Semantic Segmentation: Gives per-pixel labels, but merges instances

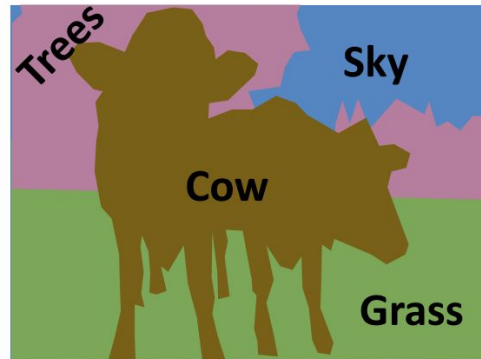


Things and Stuff

Things: Object categories that can be separated into object instances
(e.g. cats, cars, person)



[This image is CC0 public domain](#)

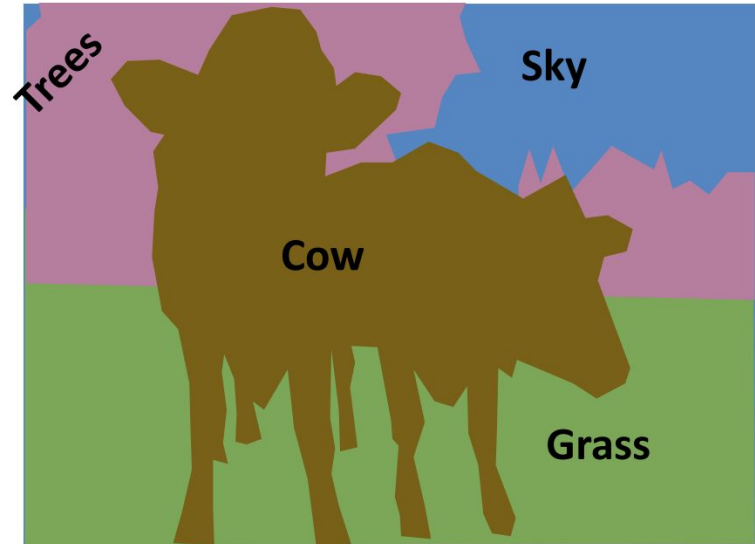


Computer Vision Tasks

Object Detection: Detects individual object instances, but only gives box (Only things!)



Semantic Segmentation: Gives per-pixel labels, but merges instances (Both things and stuff)



Some other (CV) Tasks

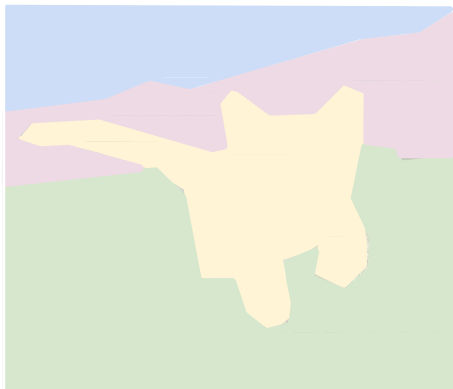
Classification



CAT

Single Object

Semantic Segmentation



GRASS, CAT,
TREE, SKY

No objects, just pixels

Object Detection



DOG, DOG, CAT

Multiple Object

Instance Segmentation



DOG, DOG, CAT

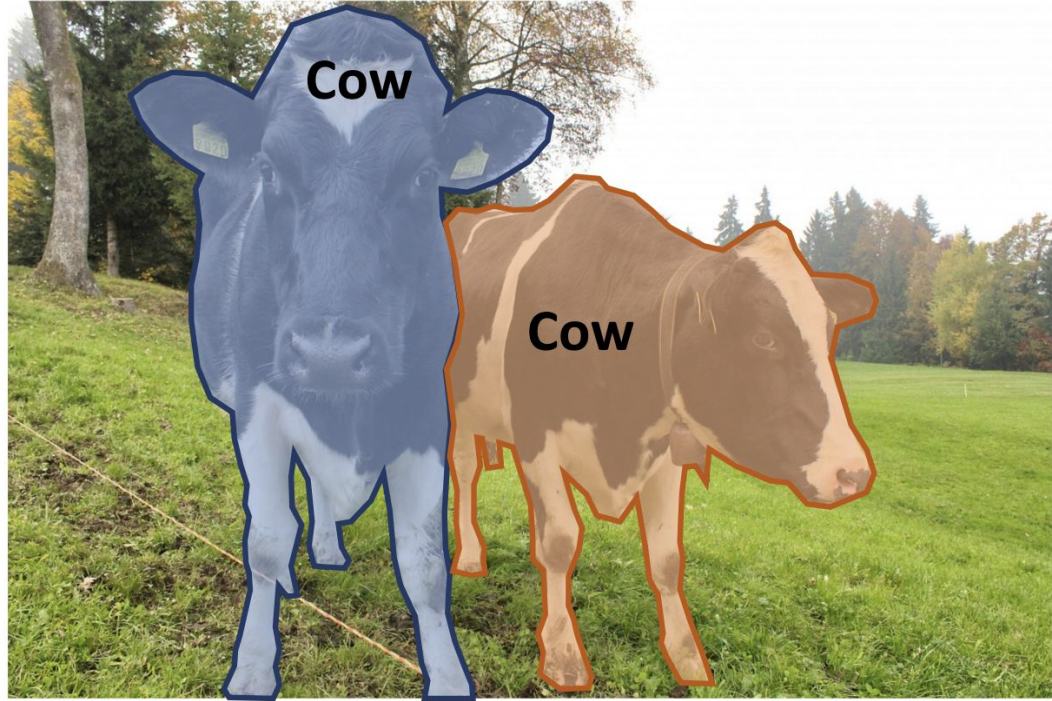
[This image is CC0 public domain](#)

Computer Vision Tasks: Instance Segmentation

Instance Segmentation:

Detect all objects in the image, and identify the pixels that belong to each object (Only things!)

Approach: Perform object detection, then predict a segmentation mask for each object!



[This image](#) is [CC0 public domain](#)

Object Detection: Faster R-CNN

Object Detection

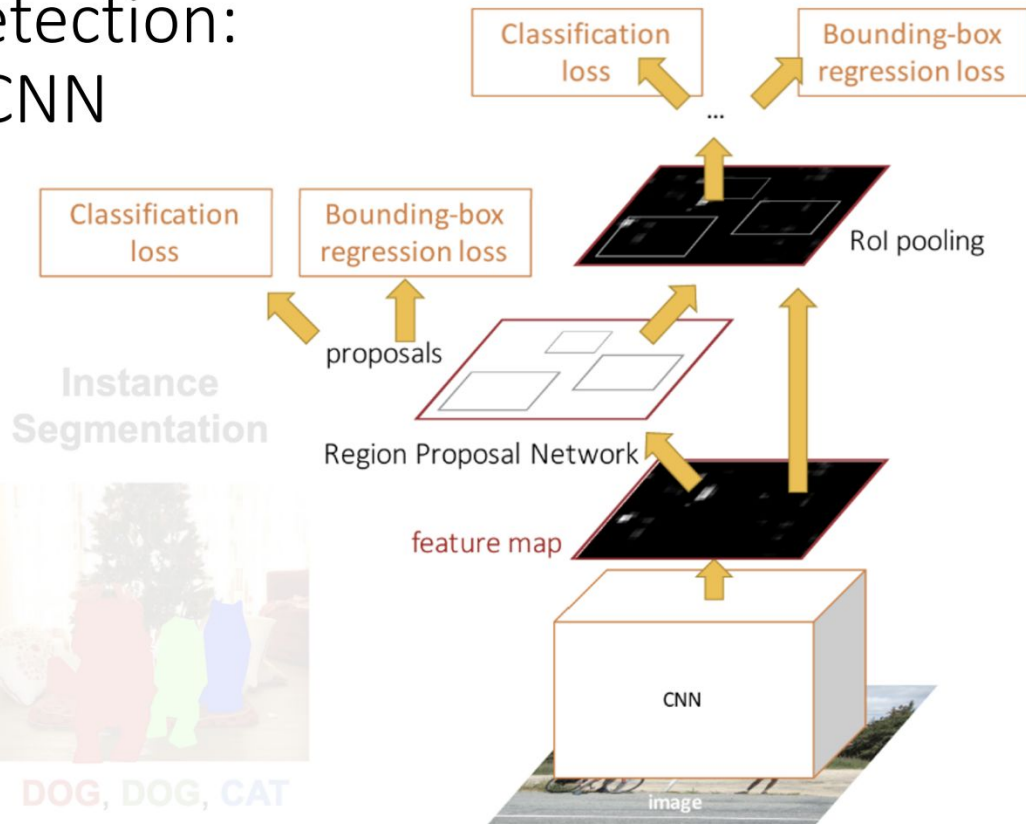


DOG, DOG, CAT

Instance Segmentation

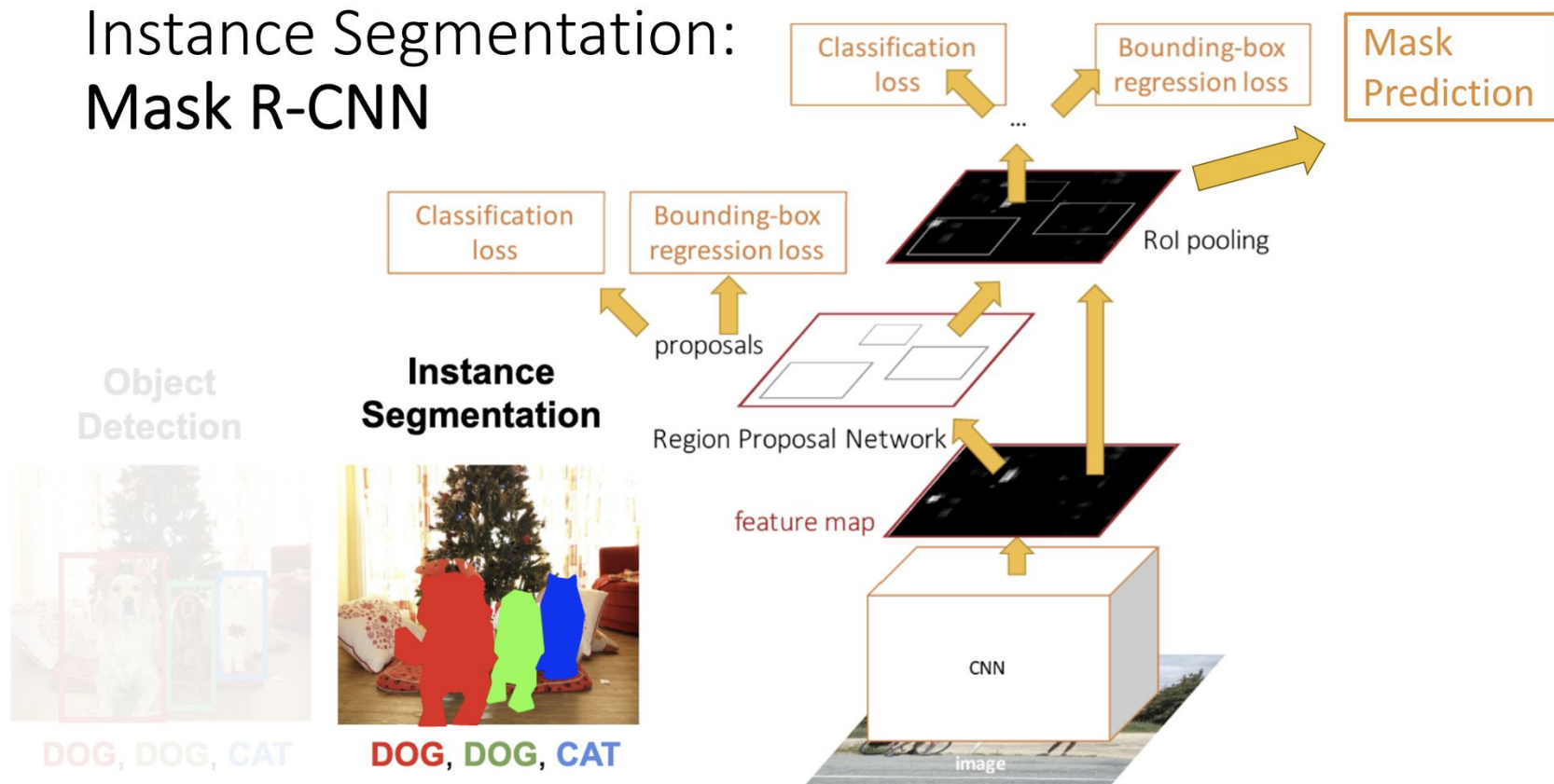


DOG, DOG, CAT



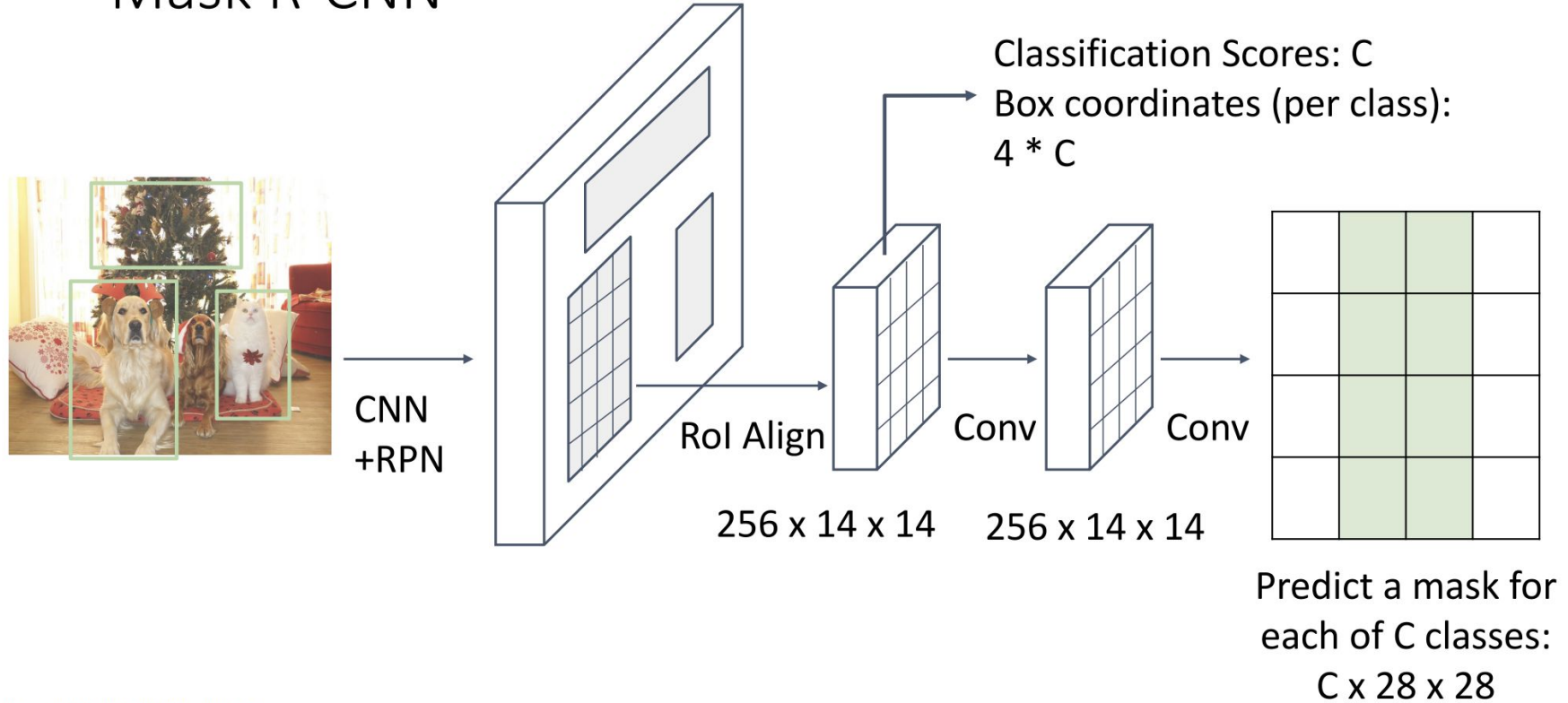
Ren et al, "Faster R-CNN: Towards Real-Time Object Detection with Region Proposal Networks", NeurIPS 2015

Instance Segmentation: Mask R-CNN



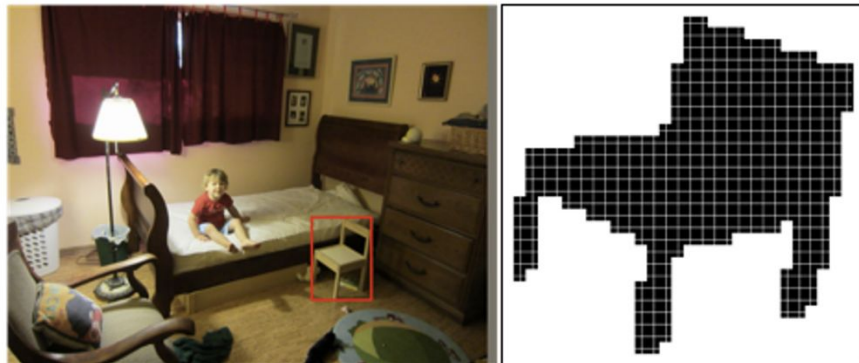
He et al, "Mask R-CNN", ICCV 2017

Mask R-CNN

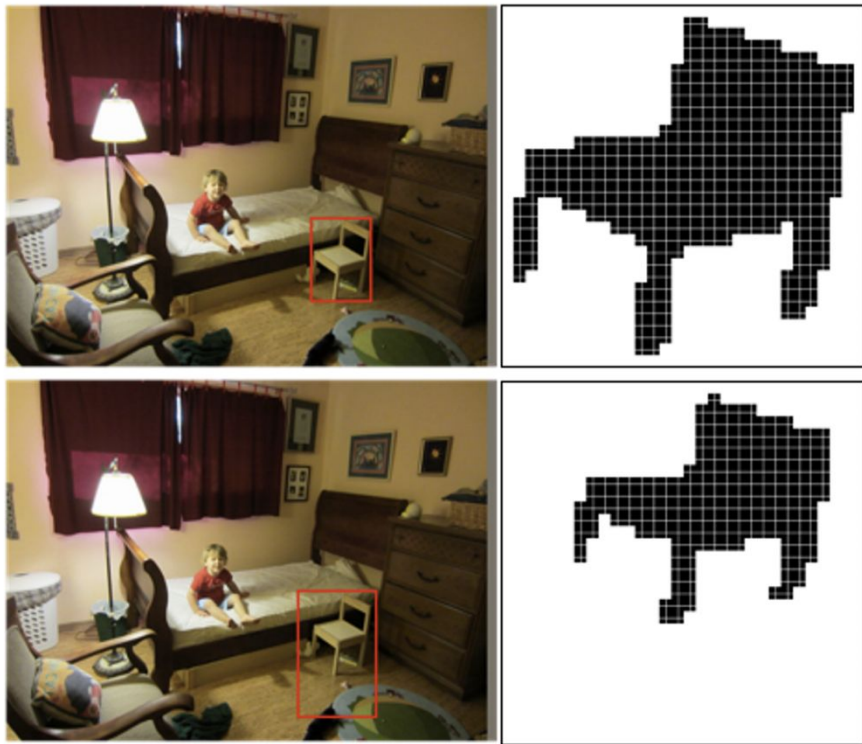


He et al. "Mask R-CNN". ICCV 2017

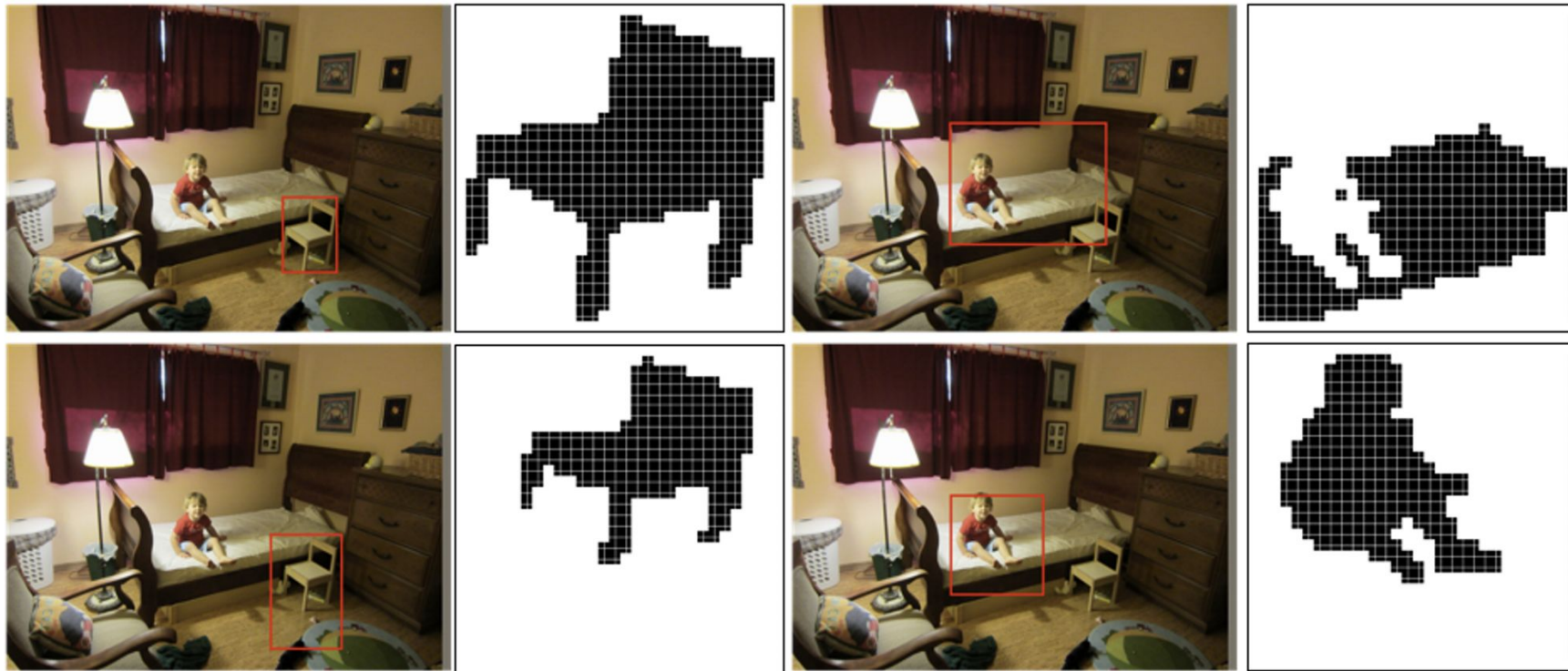
Mask R-CNN: Example Training Targets



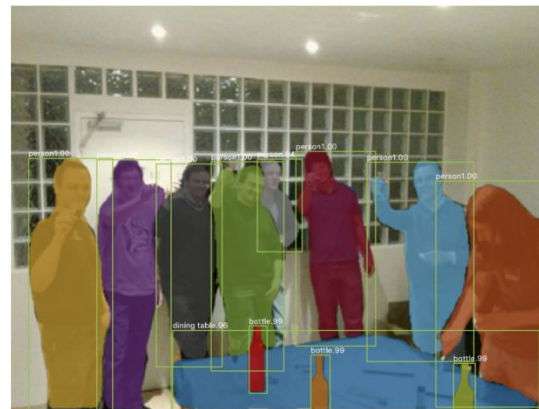
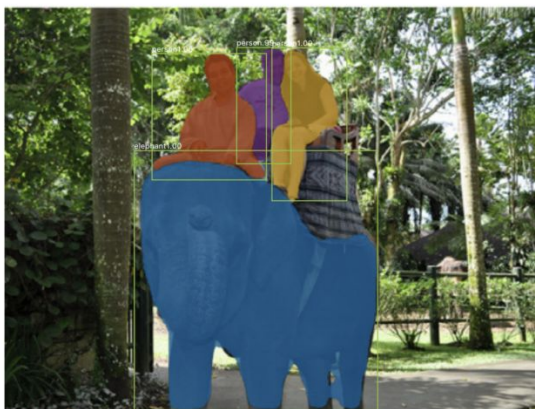
Mask R-CNN: Example Training Targets



Mask R-CNN: Example Training Targets

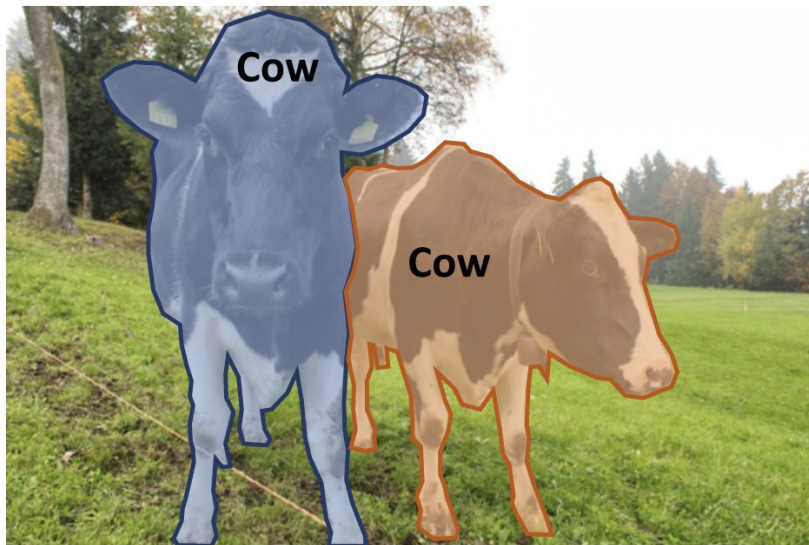


Mask R-CNN: Very Good Results!

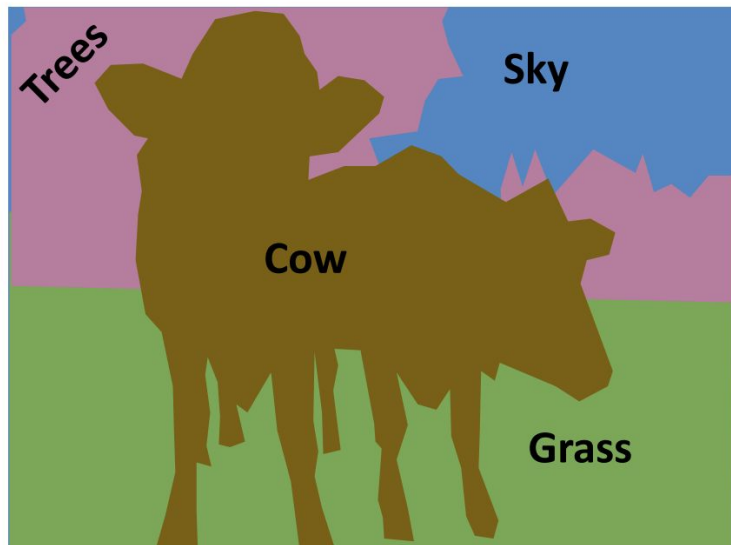


Beyond Instance Segmentation

Instance Segmentation: Separate object instances, but only things



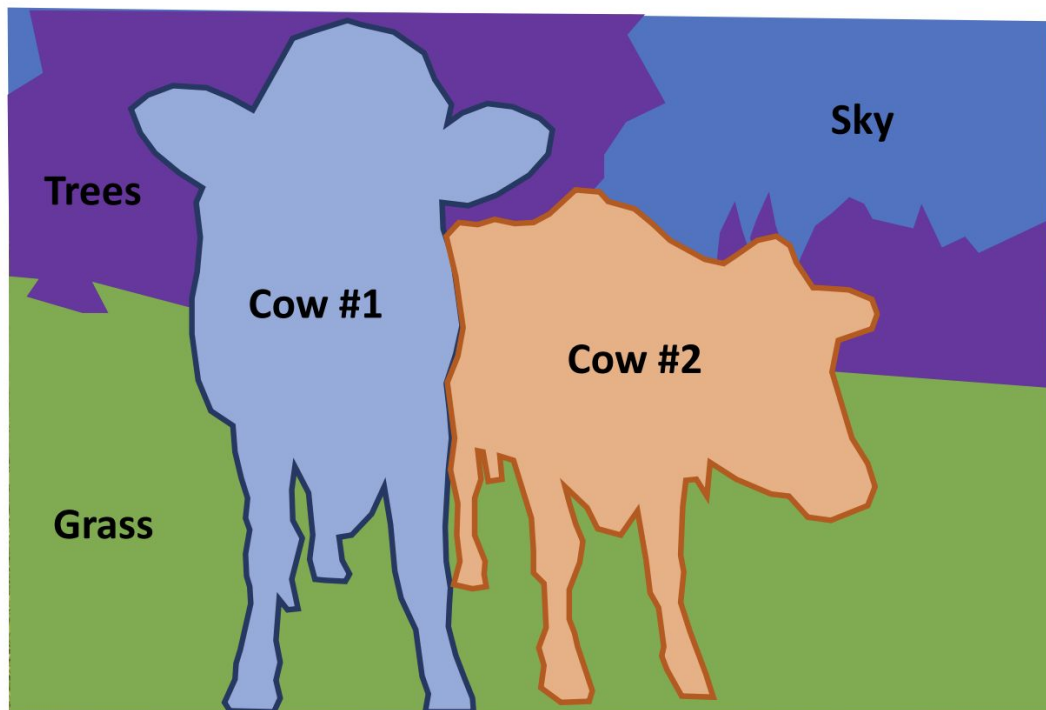
Semantic Segmentation: Identify both things and stuff, but doesn't separate instances



Beyond Instance Segmentation: Panoptic Segmentation

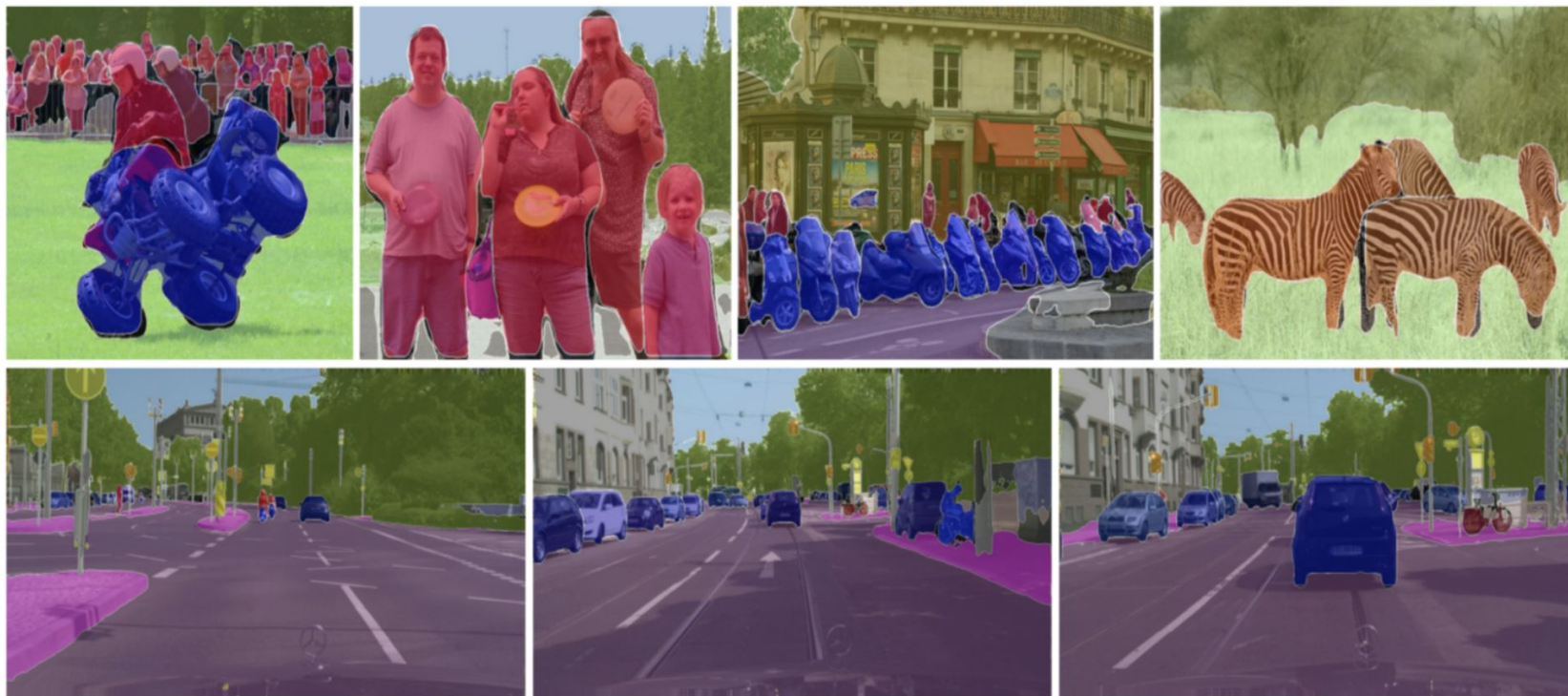
Label all pixels in the image (both things and stuff)

For “thing” categories also separate into instances



Kirillov et al, “Panoptic Segmentation”, CVPR 2019
Kirillov et al, “Panoptic Feature Pyramid Networks”, CVPR 2019

Beyond Instance Segmentation: Panoptic Segmentation



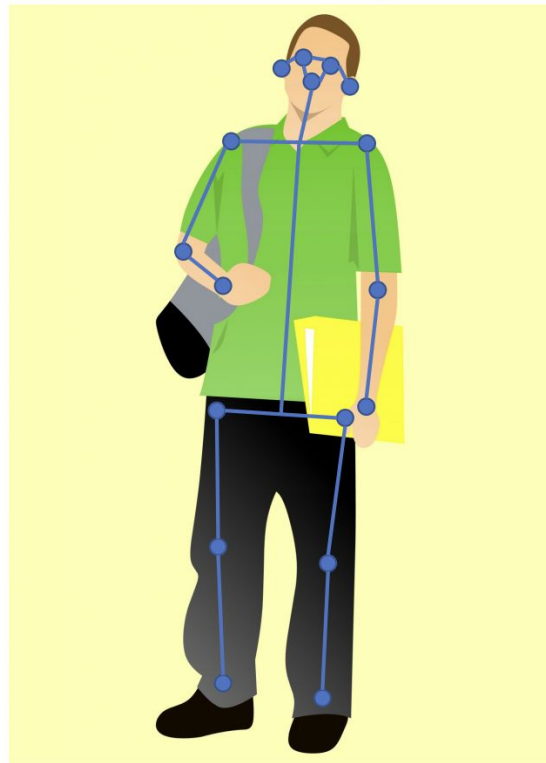
Kirillov et al, "Panoptic Feature Pyramid Networks", CVPR 2019

Beyond Instance Segmentation: Human Keypoints

Represent the pose of a human
by locating a set of **keypoints**

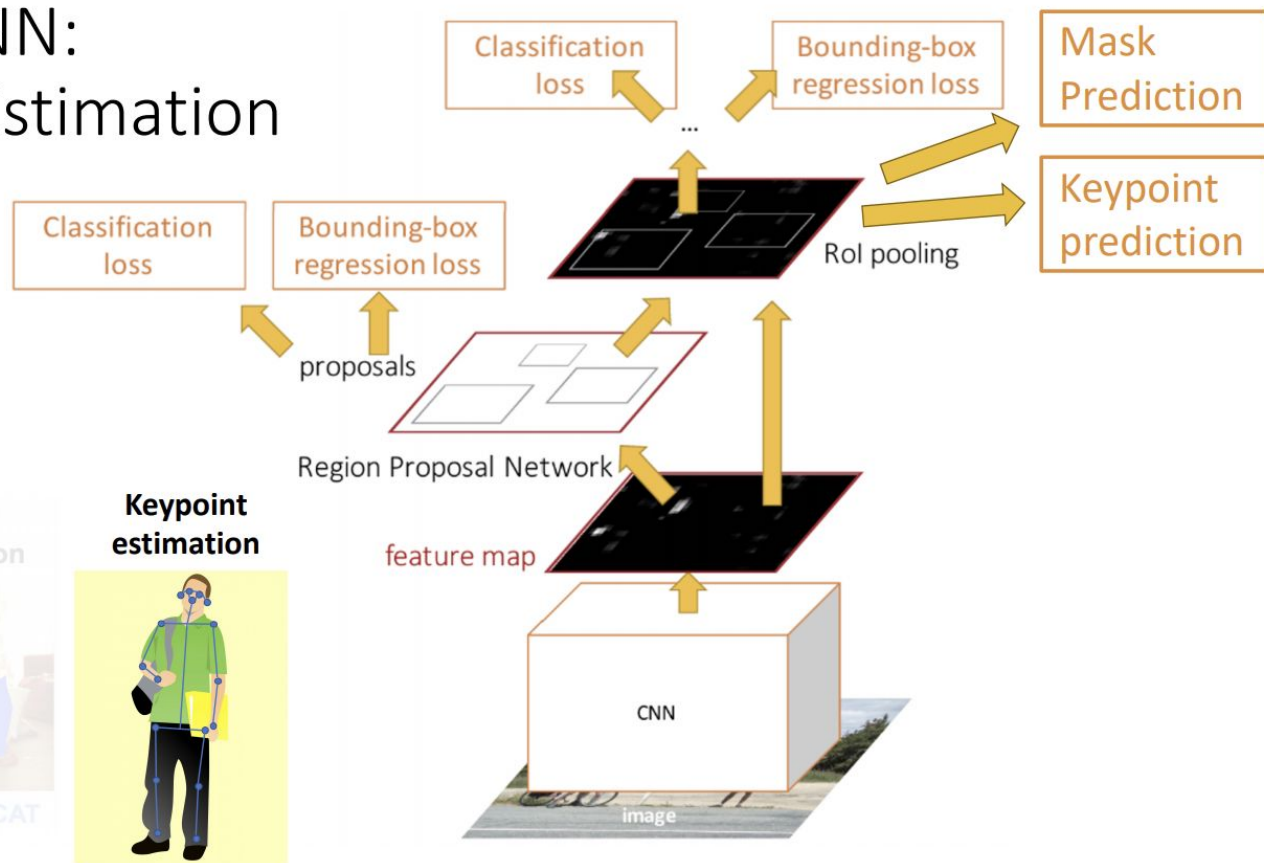
e.g. 17 keypoints:

- Nose
- Left / Right eye
- Left / Right ear
- Left / Right shoulder
- Left / Right elbow
- Left / Right wrist
- Left / Right hip
- Left / Right knee
- Left / Right ankle



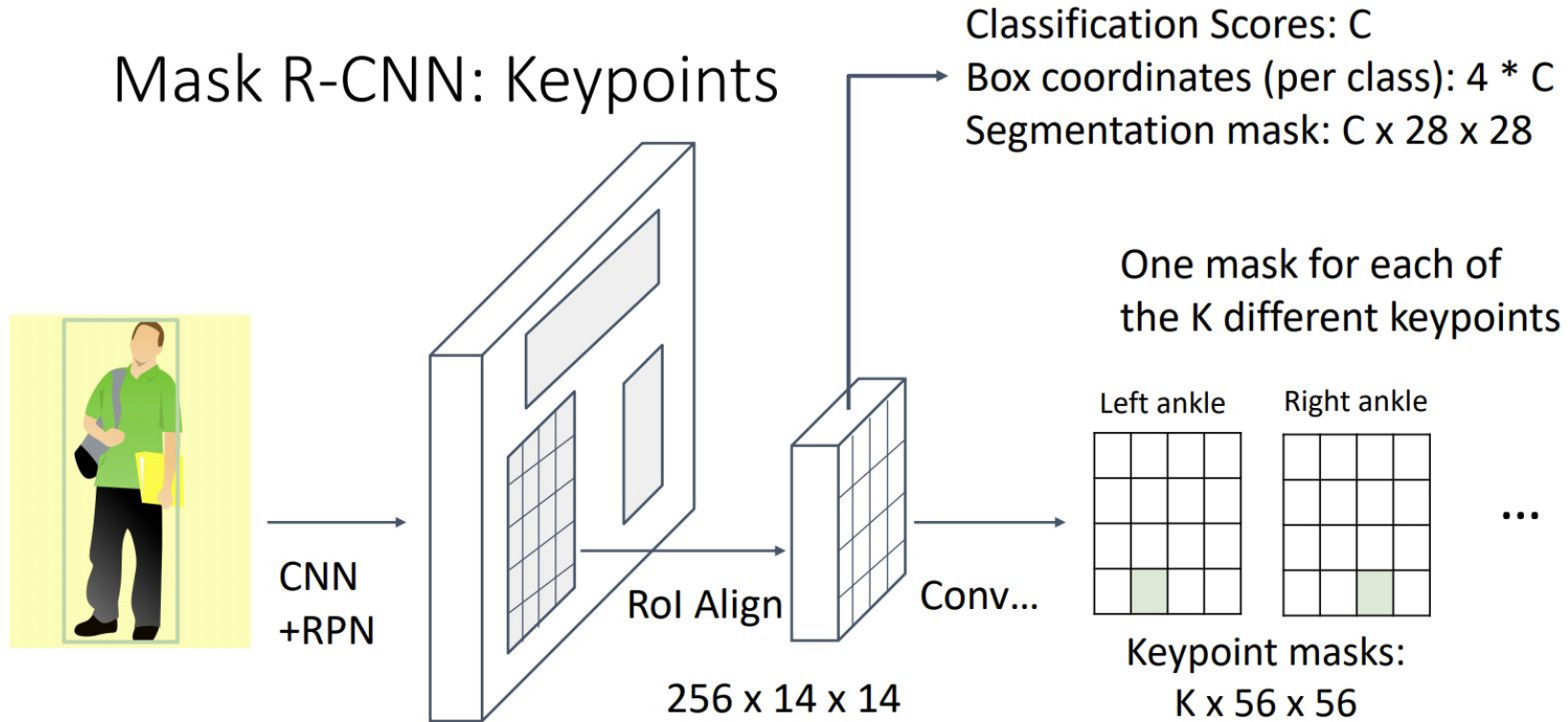
[Person image](#) is [CC0 public domain](#)

Mask R-CNN: Keypoint Estimation



He et al, "Mask R-CNN", ICCV 2017

Mask R-CNN: Keypoints



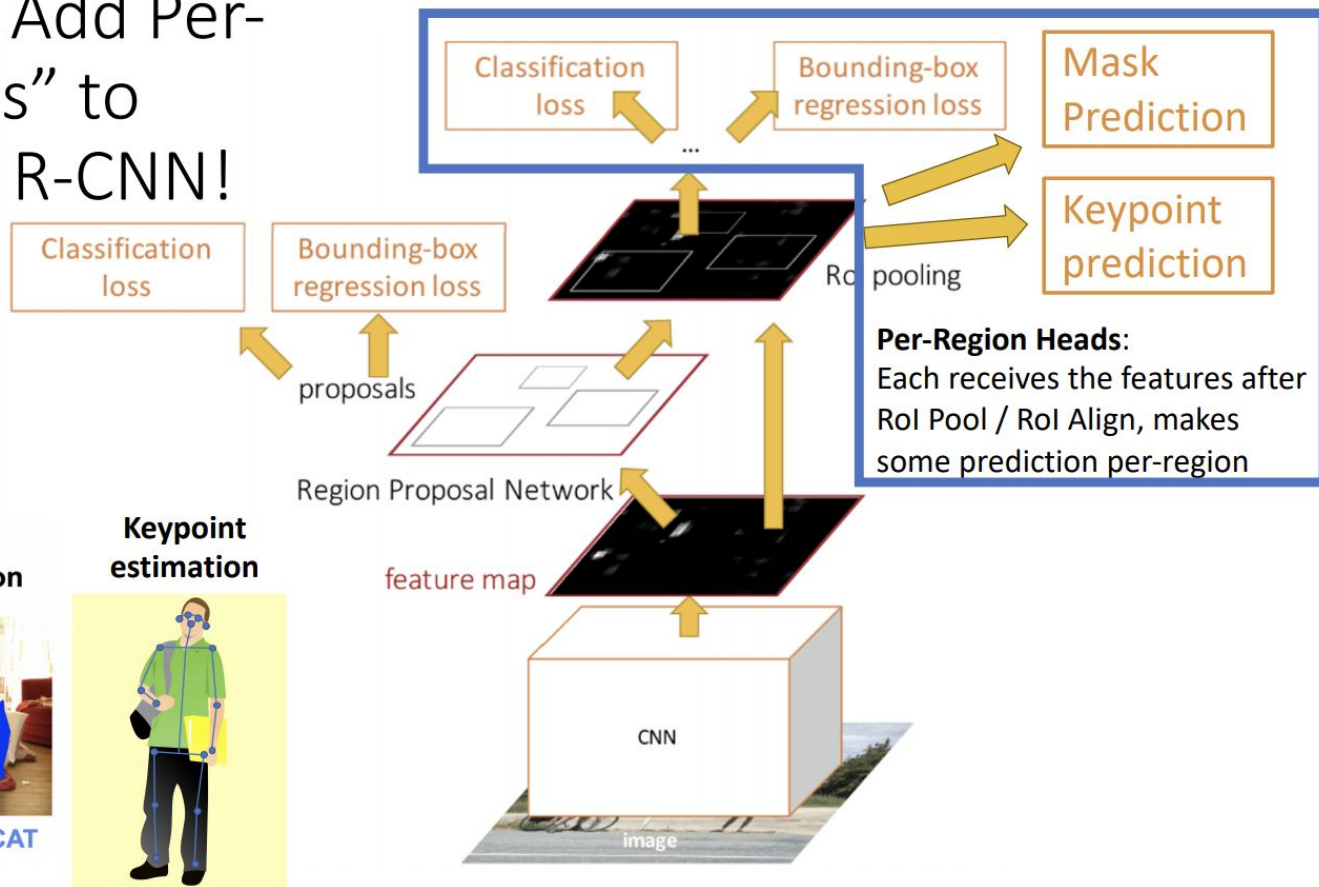
Ground-truth has one “pixel” turned on per keypoint. Train with softmax loss

Joint Instance Segmentation and Pose Estimation

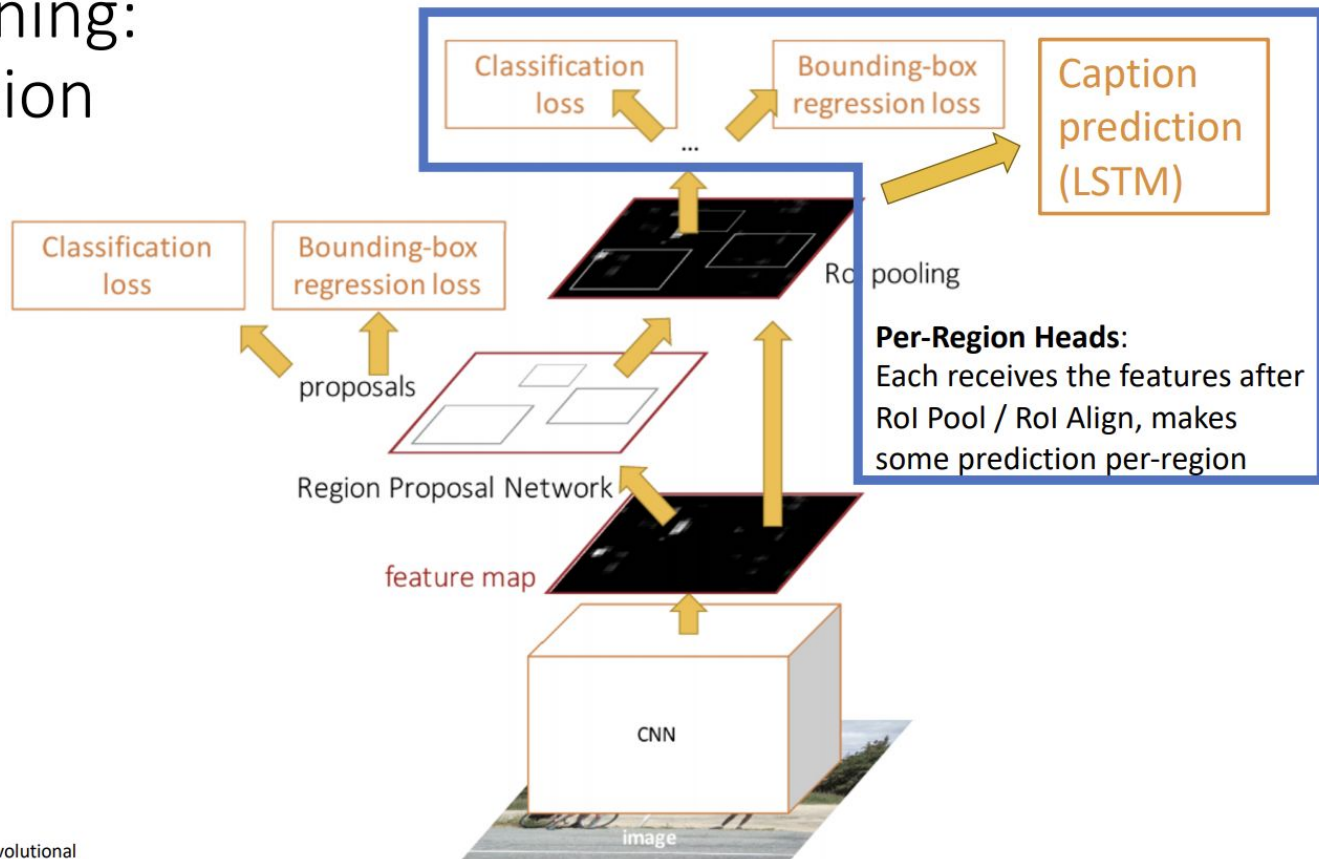


He et al, "Mask R-CNN", ICCV 2017

General Idea: Add Per-Region “Heads” to Faster / Mask R-CNN!

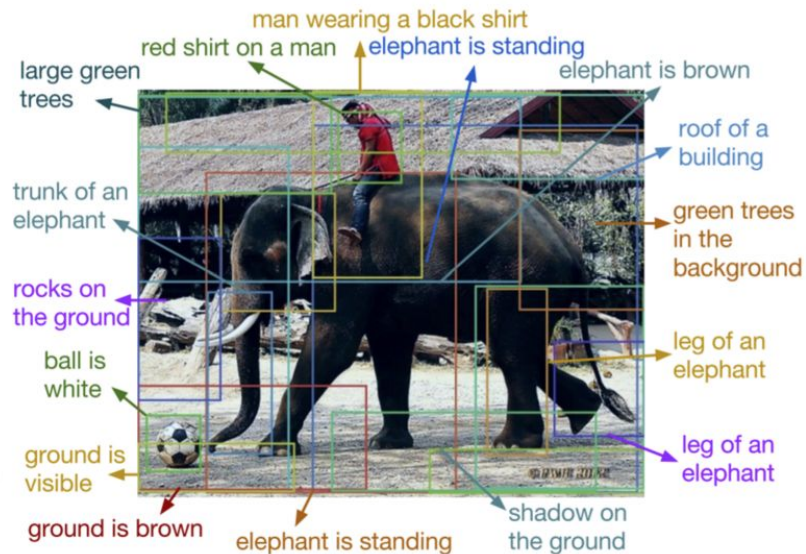
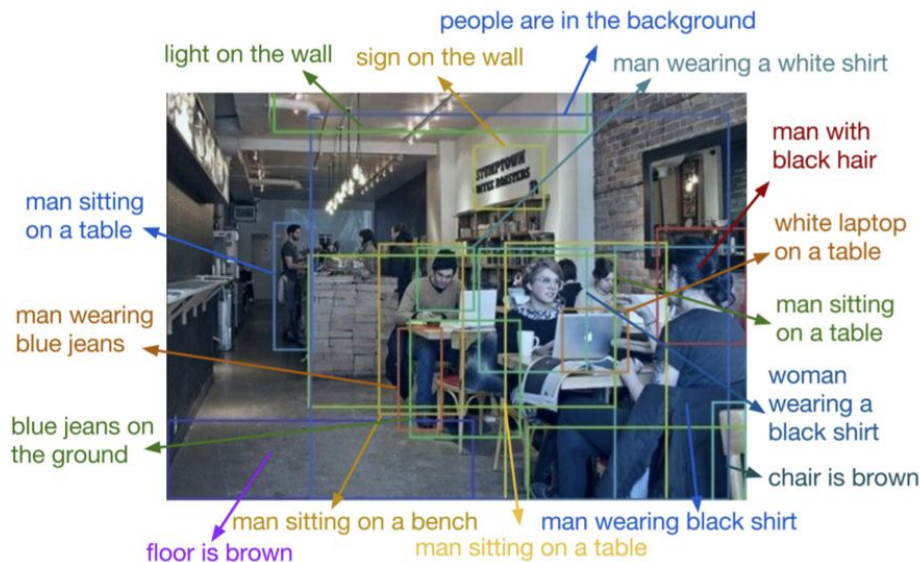


Dense Captioning: Predict a caption per region!



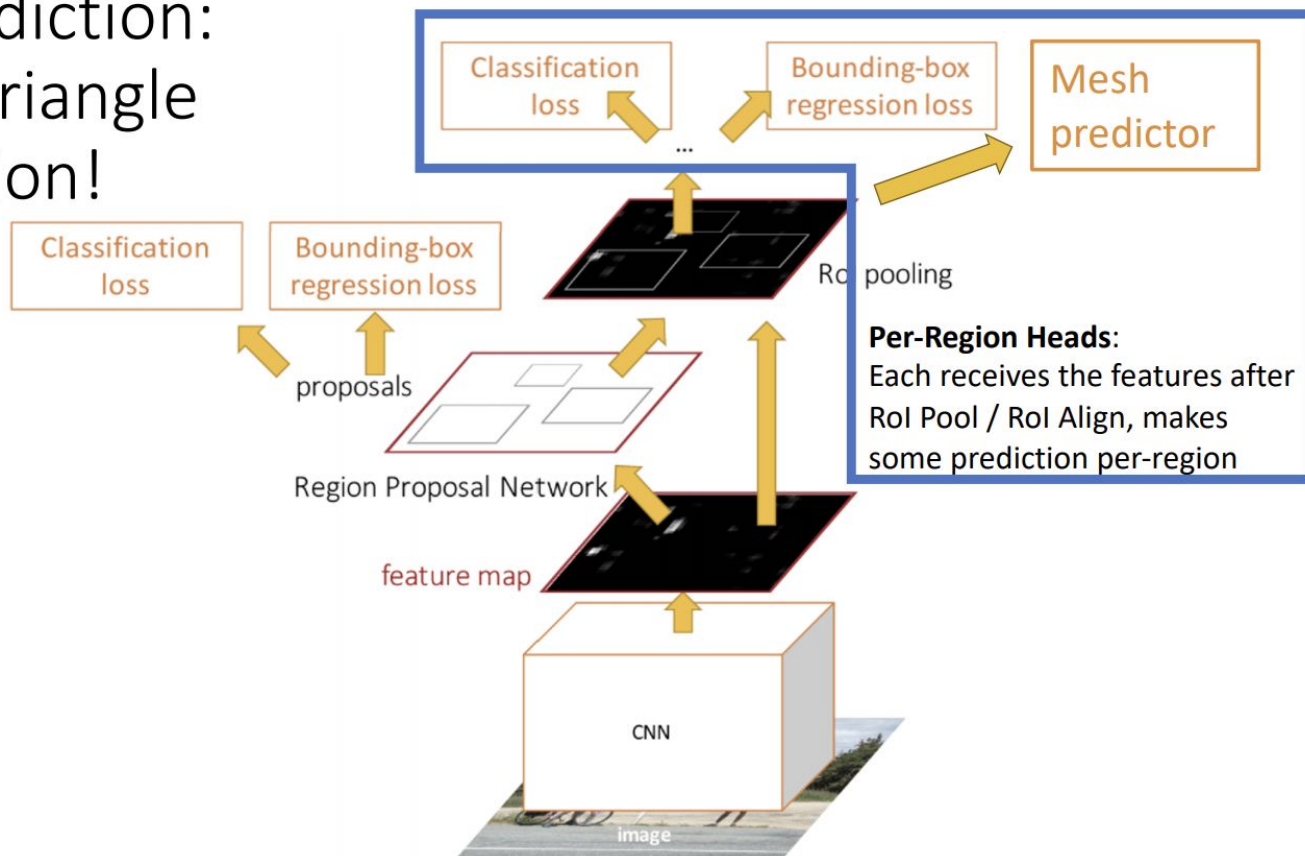
Johnson, Karpathy, and Fei-Fei, "DenseCap: Fully Convolutional Localization Networks for Dense Captioning", CVPR 2016

Dense Captioning



Johnson, Karpathy, and Fei-Fei, "DenseCap: Fully Convolutional Localization Networks for Dense Captioning", CVPR 2016

3D Shape Prediction: Predict a 3D triangle mesh per region!



3D Shape Prediction: Mask R-CNN + Mesh Head

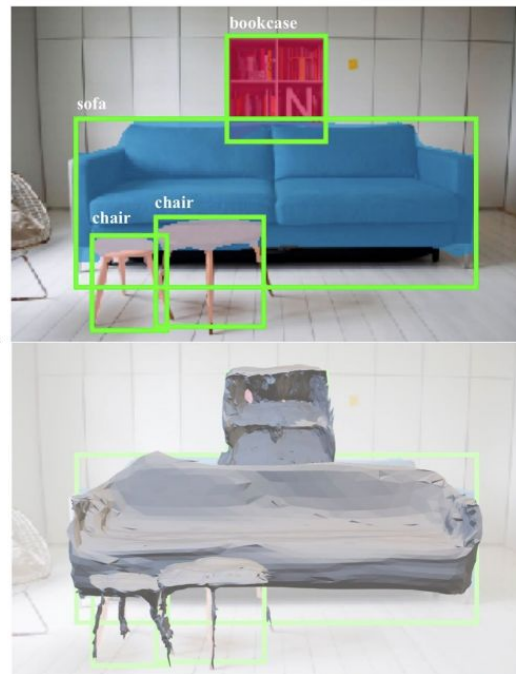
Mask R-CNN:

2D Image -> 2D shapes



Mesh R-CNN:

2D Image -> **3D** shapes



He, Gkioxari, Dollár, and Girshick, "Mask R-CNN", ICCV 2017

Gkioxari, Malik, and Johnson, "Mesh R-CNN", ICCV 2019

Some other (CV) Tasks

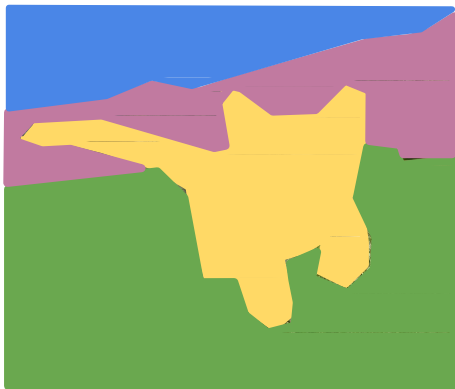
Classification



CAT

Single Object

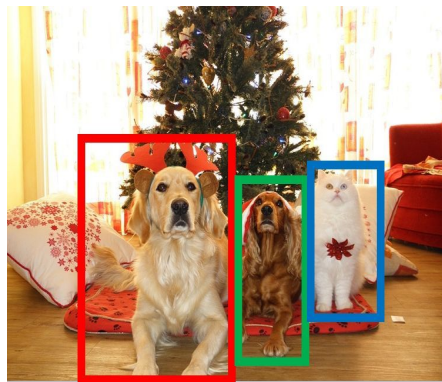
Semantic Segmentation



**GRASS, CAT,
TREE, SKY**

No objects, just pixels

Object Detection



DOG, DOG, CAT

Multiple Object

Instance Segmentation



DOG, DOG, CAT

[This image is CC0 public domain](#)